Build Your Own Mini Redis in Java

A simple, step-by-step guide to finish in 4 weeks

Real redis-cli works with it | Saves data to disk | Has a web dashboard

Written in easy language. Every hard word is explained.

1. What are you building?

Redis is a very popular program that stores data in memory (RAM) so it can be read very fast. Big companies use it to keep things like login sessions, counters and cached data. You will build your own small version of it, in Java.

The special part: it will be so correct that the **real** tool called `redis-cli` can connect to your program and use it. In a demo, this makes people say "wow" in the first 30 seconds.

What it will do at the end of 4 weeks

- Store and read text values: `SET`, `GET`, `DEL`, `INCR`
- Remove data automatically after some time (called **expiry**): `EXPIRE`, `TTL`
- Store lists and hashes (a hash is like a small table of fields and values)
- Save data to a file so nothing is lost when the program restarts
- Let many clients talk at the same time, and send messages between them (**pub/sub**)
- A good-looking **web dashboard** (Spring Boot) to see keys, run commands and watch live numbers
- A speed test (**benchmark**) against the real Redis, with a table in your README

> **How much time do I need?**
> About 2 to 3 hours per day for 28 days. Every 7th day is a light "catch up" day, so a missed day will not break your plan.

Why this project is good for your resume

- It shows you understand **how things work inside**, not only how to use a framework.
- Interviewers at product companies love questions about networking, threads, and saving data. This project gives you real answers.
- A project that works with a real tool is easy to demo and easy to believe.

2. Words made simple

Come back to this page any time a word feels heavy. Each meaning is kept short on purpose.

Word	Simple meaning
Server	A program that waits for other programs to talk to it. Yours will wait on a number called a port.
Client	The program that talks to the server. Here it is redis-cli (and later your dashboard).
Port	A door number on your computer. Your server will use port 6380.
TCP / Socket	TCP is the way two programs send data safely over a network. A socket is one connection (one phone line) between them.
Protocol	A set of rules for how to talk, like grammar. Both sides must follow it.
RESP	The grammar Redis uses. Messages are plain text with special markers. You will learn it on Day 2.
Byte / Buffer	A byte is one small piece of data. A buffer is a box in memory that holds bytes for a moment.
Parser	Code that reads raw text and understands what it means (turns bytes into a command).
Key-value store	A big dictionary. You give a key (name), it gives back the value. Like a Java HashMap.
Thread	One worker inside your program. More threads means more things done at once, but also more chances for bugs.
Concurrency	Many things happening at the same time.
Blocking vs non-blocking	Blocking: the worker waits and does nothing until data arrives. Non-blocking: the worker keeps doing other jobs and checks later.
NIO	Java's non-blocking way to do network work. One worker can look after hundreds of clients.
Event loop	One worker that runs in a circle: check who sent something, handle it, repeat.
Selector	The Java tool that tells the event loop "these clients have sent something".
TTL / Expiry	Time To Live. How many seconds a key stays before it is deleted automatically.
AOF	Append Only File. A file where you write every change at the end, so you can replay it after a restart.
fsync	A command to force the computer to really write data to the disk now, not later. Safer but slower.
Pub/Sub	Publish and Subscribe. Like a YouTube channel: subscribers get every message that is posted on the channel.
LRU	Least Recently Used. When memory is full, remove the item nobody has used for the longest time.
Pipelining	Sending many commands one after another without waiting for each answer. Makes things much faster.

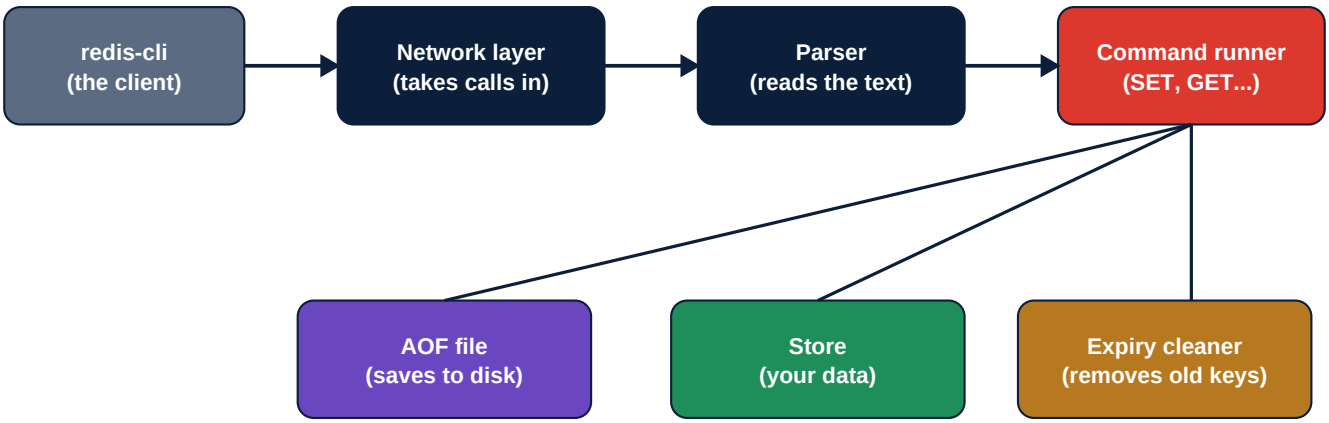
Word	Simple meaning
Benchmark	A speed test. For example: how many commands per second can my server handle?
Maven	A tool that downloads libraries and builds your Java project.
Docker	A tool that packs your program and everything it needs in a box, so it runs the same on any computer.

3. Tools to install (Day 1)

Tool	Why	How
JDK 17 or newer	To write and run Java	Install from adoptium.net, then check with `java -version`
IntelliJ IDEA (Community)	Your code editor	Free from jetbrains.com
Maven	Builds the project	IntelliJ already includes it. Choose Maven when creating the project.
Git + GitHub account	Save your work online. Commit every day.	git-scm.com
redis-cli and redis-benchmark	Real tools to test YOUR server	Mac: `brew install redis`. Ubuntu/WSL: `sudo apt install redis-tools`. Windows: use WSL or Docker.

4. The big picture

Do not worry if this looks big. You will build it one box at a time, from left to right.



The answer travels back the same way to redis-cli.

What happens when you type `SET name arpit`

- redis-cli opens a connection to port 6380 and sends the command as text.
- Your **network layer** receives the bytes.
- The **parser** understands: command is SET, key is name, value is arpit.
- The **command runner** puts it in the **store** and also writes it to the **AOF file**.
- You send back `+OK`.

5. Folder structure

Keep your code in small files, each with one job. This also looks clean on GitHub.

```
mini-redis/
  pom.xml
  README.md
  src/main/java/com/arpit/miniredis/
    Main.java                (starts the server)
    server/
      SimpleServer.java      (Week 1: easy version, one thread per client)
      NioServer.java         (Week 2: faster version, one event loop)
      ClientConnection.java  (one connected client + its buffer)
    protocol/
      RespParser.java        (reads commands)
      RespWriter.java        (builds replies)
    command/
      CommandHandler.java    (decides what each command does)
    store/
      Store.java             (the data + expiry + LRU)
    persistence/
      AofWriter.java         (save to file)
      AofLoader.java         (read file at start)
    pubsub/
      PubSub.java            (channels and subscribers)
  src/test/java/...          (JUnit tests)
  dashboard/                 (Week 4: Spring Boot web UI)
  docker-compose.yml
```

Golden rule
Always keep the program working. At the end of every day, `redis-cli -p 6380` should still work, and you should `git commit`. Small working steps beat big broken ones.

How to read the day plans

Each day has a goal and a line called **Done when**. If you can tick that line, move on. If not, use the next catch-up day. Do not skip ahead.

Week 1: Make it talk

Goal: the real redis-cli connects to your server, and `PING`, `SET`, `GET`, `DEL` work. We use the **easy** design this week: one thread per client. Next week you will upgrade it.

Day	What to do	Done when
1	Install tools. Create the Maven project. Write a tiny server using `ServerSocket(6380)` that accepts a connection and prints every byte it receives. Run `redis-cli -p 6380 PING`.	You see the raw text of the command printed in your console.
2	Learn RESP (below). Write `RespParser.readCommand()` that turns the raw text into a Java list like [SET, name, arpit].	Console prints the list for any command you type in redis-cli.
3	Write `RespWriter` (builds replies). Add `PING` and `ECHO`.	redis-cli PING prints PONG.
4	Create `Store` with a map. Add `SET`, `GET`, `DEL`, `EXISTS` inside `CommandHandler`.	SET name arpit, then GET name returns arpit.
5	Add `INCR`, `DECR`, `DBSIZE`, `KEYS *`, `FLUSHALL`. Return proper errors for wrong input and unknown commands.	INCR on a text value gives an error, not a crash.
6	Make sure two redis-cli windows work at the same time. Write JUnit tests for the parser and the store.	Tests are green. Two clients share the same data.
7	Catch-up day. Push to GitHub. Start the README.	Code is on GitHub with a short README.

RESP in 2 minutes (Day 2)

When you type `SET name arpit`, redis-cli sends this text. Each line ends with `\r\n` (two invisible characters meaning "new line").

```
*3        <- "an array of 3 items is coming"
$3        <- "next item is 3 characters long"
SET
$4        <- "next item is 4 characters long"
name
$5
arpit
```

And your server answers using one of these starters:

Reply starts with	Meaning	Example
`+`	Simple text	`+OK`
`-`	Error	`-ERR unknown command`
`:`	A number	`:1`
`$`	Text with its length	`$5` then hello
`$-1`	Nothing found (null)	GET on a missing key
`*`	A list of items	`*2` then two items

Week 1 code you can start from

Day 1 and 2 - the simple server and the parser. Type it yourself instead of copying. You will remember it much better.

```
// SimpleServer.java - one thread per client
public void start() throws IOException {
    ServerSocket server = new ServerSocket(6380);
    System.out.println("Mini Redis listening on 6380");
    while (true) {
        Socket client = server.accept();          // waits for a client
        new Thread(() -> handle(client)).start(); // one worker per client
    }
}

private void handle(Socket client) {
    try (client;
         InputStream in = new BufferedInputStream(client.getInputStream());
         OutputStream out = client.getOutputStream()) {
        List<String> cmd;
        while ((cmd = RespParser.readCommand(in)) != null) {
            String reply = handler.run(cmd);      // CommandHandler decides
            out.write(reply.getBytes(StandardCharsets.UTF_8));
            out.flush();
        }
    } catch (IOException e) { /* client left, that is fine */ }
}
```

```
// RespParser.java - reads ONE command from the stream
public static List<String> readCommand(InputStream in) throws IOException {
    String first = readLine(in);
    if (first == null) return null;               // client disconnected
    int count = Integer.parseInt(first.substring(1));   // "*3" -> 3
    List<String> parts = new ArrayList<>();
    for (int i = 0; i < count; i++) {
        String lenLine = readLine(in);            // "$3"
        int len = Integer.parseInt(lenLine.substring(1));
        byte[] data = in.readNBytes(len);         // the real text
        in.readNBytes(2);                         // skip the \r\n
        parts.add(new String(data, StandardCharsets.UTF_8));
    }
    return parts;
}

private static String readLine(InputStream in) throws IOException {
    StringBuilder sb = new StringBuilder();
    int b;
    while ((b = in.read()) != -1) {
        if (b == '\r') { in.read(); return sb.toString(); }  // skip \n too
        sb.append((char) b);
    }
    return null;
}
```

Day 3-4 - replies and the command handler.

```
// RespWriter.java
public static String ok()              { return "+OK\r\n"; }
public static String error(String m)   { return "-ERR " + m + "\r\n"; }
public static String integer(long n)   { return ":" + n + "\r\n"; }
public static String bulk(String s) {
    if (s == null) return "$-1\r\n";                      // null
    return "$" + s.getBytes(StandardCharsets.UTF_8).length + "\r\n" + s + "\r\n";
}

// CommandHandler.java
public String run(List<String> cmd) {
    String name = cmd.get(0).toUpperCase();
    switch (name) {
        case "PING": return "+PONG\r\n";
        case "SET":  store.set(cmd.get(1), cmd.get(2)); return RespWriter.ok();
        case "GET":  return RespWriter.bulk(store.get(cmd.get(1)));
        case "DEL":  return RespWriter.integer(store.del(cmd.get(1)) ? 1 : 0);
        default:     return RespWriter.error("unknown command '" + name + "'");
    }
}
```

> **Tip**
> When redis-cli starts, newer versions also send a command called `COMMAND DOCS`. If your server answers with an error, that is fine. It still works.

Week 2: Make it fast and smart

Goal: replace the one-thread-per-client design with **one event loop** (this is how real Redis works), then add expiry, lists and hashes.

> **Why change the design?**
> Thread per client is easy but wasteful: 1,000 clients means 1,000 threads. With an event loop, **one** worker handles everyone. It is faster, it needs no locks, and "why is Redis single-threaded and still fast?" is a famous interview question.

Day	What to do	Done when
8	Learn NIO. Build a tiny echo server using `Selector`, `ServerSocketChannel` and `SocketChannel` (code below).	Two clients type text and get it echoed back.
9	Make the parser work on a `ByteBuffer`. It must return **"not complete yet"** if only half a command has arrived, and wait for more bytes.	Unit test: feed a command in 3 pieces, get one result.
10	Plug `CommandHandler` into the event loop. Now only one thread touches data, so change the map to a plain `HashMap`.	All Week 1 commands still work.
11	Add expiry: `EXPIRE`, `TTL`, `PERSIST`, `SET key value EX 10`. Use **lazy delete** (check the time when a key is read).	A key disappears 10 seconds after SET ... EX 10.
12	Add **active delete**: every 100 ms, check a small batch of keys with a timer and remove expired ones. Use `selector.select(100)` so the loop wakes up on time.	Keys nobody reads still vanish.
13	Lists: `LPUSH`, `RPUSH`, `LPOP`, `RPOP`, `LRANGE`, `LLEN`. Store a Java `ArrayDeque` as the value. Return a WRONGTYPE error if the key holds a different type.	LPUSH then LRANGE 0 -1 shows the list.
14	Hashes: `HSET`, `HGET`, `HDEL`, `HGETALL`. Catch-up and tests.	All tests green. Push to GitHub.

The event loop skeleton (Day 8)

Read this slowly. The **Selector** is like a receptionist: it tells your one worker which clients have sent something.

```
Selector selector = Selector.open();
ServerSocketChannel server = ServerSocketChannel.open();
server.bind(new InetSocketAddress(6380));
server.configureBlocking(false);                    // non-blocking
server.register(selector, SelectionKey.OP_ACCEPT);  // "tell me about new clients"

while (true) {
    selector.select(100);                           // wait up to 100 ms
    Iterator<SelectionKey> it = selector.selectedKeys().iterator();
    while (it.hasNext()) {
        SelectionKey key = it.next();
        it.remove();
        if (key.isAcceptable()) {                   // a new client arrived
            SocketChannel ch = server.accept();
            ch.configureBlocking(false);
            ch.register(selector, SelectionKey.OP_READ, new ClientConnection(ch));
        } else if (key.isReadable()) {              // a client sent bytes
            ((ClientConnection) key.attachment()).onReadable();
        }
    }
    store.removeSomeExpiredKeys();                  // active expiry (Day 12)
}
```

Partial data: the most important idea of Week 2

Network data arrives like water in a pipe, not in neat packages. Your server may receive **half** a command, or **two commands glued together**. Your code must handle both.

```
// ClientConnection.java - each client has its own buffer
void onReadable() throws IOException {
    int n = channel.read(readBuffer);          // add new bytes to the end
    if (n == -1) { close(); return; }          // client left
    readBuffer.flip();                         // switch buffer to "reading mode"
    while (true) {
        int startPos = readBuffer.position();
        List<String> cmd = RespParser.tryParse(readBuffer);  // null if incomplete
        if (cmd == null) {
            readBuffer.position(startPos);     // undo: we did not use these bytes
            break;                             // wait for more data
        }
        write(handler.run(cmd));               // one complete command: run it
    }
    readBuffer.compact();                      // keep unused bytes for next time
}
```

How to test this
Use a tool called `nc` (netcat) or write a JUnit test that sends `*1\r\n$4\r\nPI`, waits, then sends `NG\r\n`. You must still get PONG. Later, `redis-benchmark -P 16` will test the glued-together case for you.

Lazy and active expiry (Day 11-12)

Real Redis uses two ways together, and so will you:

- **Lazy:** when someone asks for a key, first check if its time is over. If yes, delete it and say "not found".
- **Active:** a small timer job looks at some keys every 100 ms and removes the dead ones. This cleans keys nobody asks for.

```
// Store.java
private final Map<String, Object> data = new HashMap<>();
private final Map<String, Long> expireAt = new HashMap<>();  // key -> time in ms

public Object get(String key) {
    Long t = expireAt.get(key);
    if (t != null && System.currentTimeMillis() > t) {   // lazy delete
        data.remove(key);
        expireAt.remove(key);
        return null;
    }
    return data.get(key);
}

public void expire(String key, long seconds) {
    expireAt.put(key, System.currentTimeMillis() + seconds * 1000);
}
```

Week 3: Make it safe and useful

Goal: your data survives a restart, clients can send messages to each other, and you have speed numbers to show.

Day	What to do	Done when
15	**AOF:** after every write command (SET, DEL, EXPIRE, LPUSH...), append the same RESP text to a file called `appendonly.aof`. Do not log reads like GET.	The file grows as you type commands.
16	**Replay:** at startup, read the AOF file and feed it through your parser and handler (without writing it again). Add the setting `always / everysec / no` for fsync.	Restart the server and your keys are still there.
17	**Pub/Sub:** `SUBSCRIBE`, `UNSUBSCRIBE`, `PUBLISH`. Keep a map from channel name to a list of connections.	Window A subscribes. Window B publishes. A receives the message.
18	Add `INFO` (uptime, number of keys, total commands), and small stubs for `COMMAND` and `CONFIG GET` (return an empty list) so real tools do not complain.	redis-benchmark runs without errors.
19	**Optional - LRU:** add a `maxKeys` setting. Use `LinkedHashMap` with access order. When too many keys, remove the oldest unused one.	With maxKeys=3, adding a 4th key removes the least-used one.
20	**Benchmark:** run the speed test (below). Find the slowest part and improve it (for example, reuse buffers, avoid many small strings).	You have numbers saved in a table.
21	Catch-up day. More JUnit tests, especially for AOF and expiry. Push to GitHub.	Tests green. README has a features list.

AOF idea in one picture

The trick: save commands in the **same text format** they arrived in. When the server starts, you just "replay" the file like a tape, using the parser you already wrote. No new code to learn.

```
// AofWriter.java
public void log(List<String> cmd) throws IOException {
    out.write(RespWriter.array(cmd).getBytes(StandardCharsets.UTF_8));
    if (policy == Policy.ALWAYS) out.getFD().sync();   // fsync every time: safest, slowest
}
// "everysec": call out.getFD().sync() from a timer once per second
// "no":       never call sync(), the computer decides

// AofLoader.java - at startup
try (InputStream in = new BufferedInputStream(new FileInputStream("appendonly.aof"))) {
    List<String> cmd;
    while ((cmd = RespParser.readCommand(in)) != null) {
        handler.runWithoutLogging(cmd);
    }
}
```

Speed test (Day 20)

```
# start your server, then in another terminal:
redis-benchmark -p 6380 -t set,get -n 100000 -c 50 -q
redis-benchmark -p 6380 -t set,get -n 100000 -c 50 -P 16 -q   # with pipelining

# run the same on real Redis (port 6379) and compare
```

Test	My Mini Redis	Real Redis
SET (requests/sec)	fill in	fill in
GET (requests/sec)	fill in	fill in
SET with pipelining	fill in	fill in

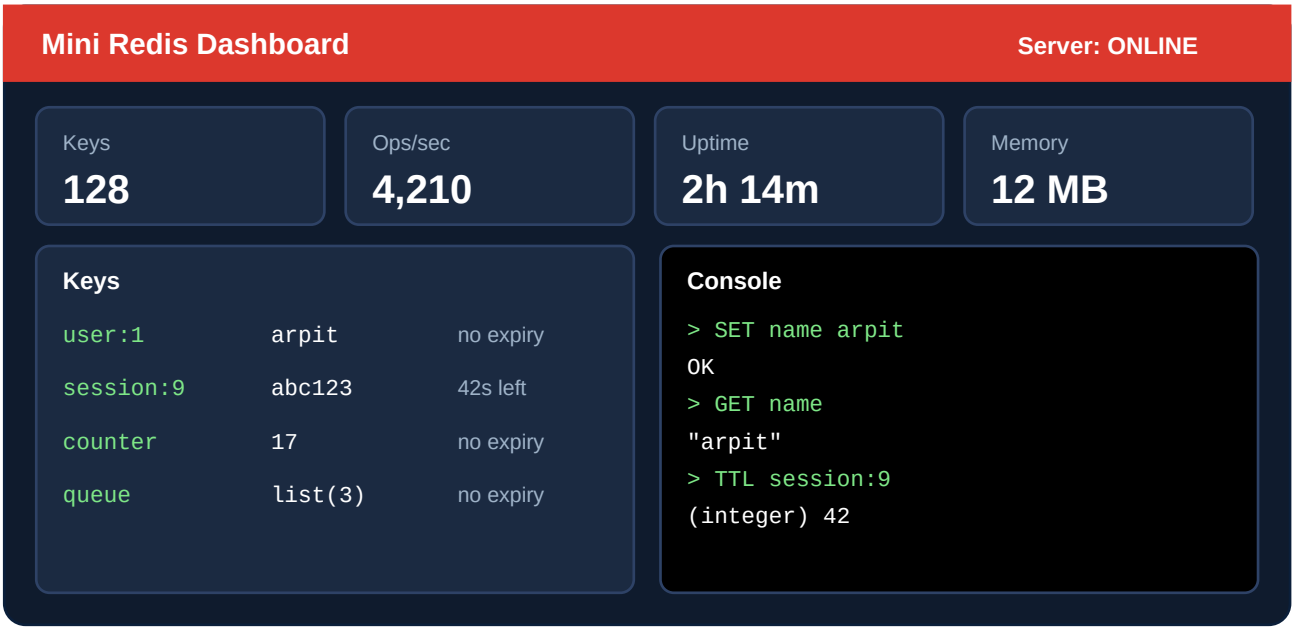
You do not need to beat real Redis. Being in the same range is already impressive, and being honest about the numbers earns trust.

Week 4: Make it look great

Goal: a web dashboard, Docker, and a README that makes people stop scrolling.

Day	What to do	Done when
22	Create a Spring Boot project in the `dashboard/` folder (Spring Web). Write `MiniRedisClient`: it opens a socket to port 6380, sends a command, reads the reply.	A Java test sends PING and gets PONG.
23	Create the web API: `POST /api/command` (run any command), `GET /api/keys` (uses KEYS *), `GET /api/info` (uses INFO).	Opening /api/info in the browser shows JSON.
24	Build the page `index.html` in `static/`: a **console box** to type commands and a **table of keys** with their values and time left.	You can run SET and see the key appear in the table.
25	Add 4 stat cards (keys, ops/sec, uptime, memory) that refresh every 2 seconds. Optional: a small live chart with Chart.js.	Numbers change while you run redis-benchmark.
26	Write a `Dockerfile` for the server and a `docker-compose.yml` that starts the server and dashboard together.	One command `docker compose up` starts everything.
27	Write the final README (see Page 17). Record a short GIF or video. Add the benchmark table. Tag release `v1.0`.	Someone new can run it in 5 minutes by following your README.
28	Resume bullets, LinkedIn post, interview practice (next pages).	You can explain every part in simple words.

What the dashboard will look like



Use a dark background, one accent color (Redis red works well), and a monospace font for keys and the console. A clean, simple design looks more professional than a colorful one.

Dashboard code to start from

Day 22-23. The dashboard is a normal Spring Boot app. It talks to your Mini Redis the same way redis-cli does.

```
// MiniRedisClient.java
@Component
public class MiniRedisClient {
    public synchronized String send(String... parts) throws IOException {
        try (Socket s = new Socket("localhost", 6380)) {
            OutputStream out = s.getOutputStream();
            StringBuilder sb = new StringBuilder("*" + parts.length + "\r\n");
            for (String p : parts) {
                sb.append("$").append(p.getBytes(StandardCharsets.UTF_8).length)
                  .append("\r\n").append(p).append("\r\n");
            }
            out.write(sb.toString().getBytes(StandardCharsets.UTF_8));
            out.flush();
            return new BufferedReader(new InputStreamReader(s.getInputStream())).readLine();
        }
    }
}

// ApiController.java
@RestController
@RequestMapping("/api")
public class ApiController {
    private final MiniRedisClient client;
    public ApiController(MiniRedisClient client) { this.client = client; }

    @PostMapping("/command")
    public Map<String, String> run(@RequestBody Map<String, String> body) throws IOException {
        String[] parts = body.get("command").trim().split("\\s+");
        return Map.of("reply", client.send(parts));
    }
}
```

> **Note**
> The code above reads only the **first line** of the reply. That works for OK, numbers and single values. For lists (like KEYS), you need to read the full RESP reply. Reuse your own `RespParser` for this. It is a good exercise.

The page (very short version)

```
<input id="cmd" placeholder="SET name arpit">
<pre id="out"></pre>
<script>
document.getElementById("cmd").addEventListener("keydown", async e => {
  if (e.key !== "Enter") return;
  const res = await fetch("/api/command", {method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify({command: e.target.value})});
  const data = await res.json();
  document.getElementById("out").textContent += "> " + e.target.value + "\n" + data.reply + "\n";
  e.target.value = "";
});
</script>
```

The README that gets attention

HR and recruiters often look at your README for 20 seconds. Put these in this order:

- **One-line pitch:** "A Redis-compatible in-memory database built from scratch in Java."
- **A GIF** of redis-cli working against your server, and a screenshot of the dashboard.
- **Features list** with ticks.
- **Architecture picture** (the one on Page 5 is a good start).
- **Run in 1 command:** `docker compose up`
- **Benchmark table** with your numbers.
- **Design decisions:** 5 short lines, such as "Why one thread?" and "Why AOF?".
- **What I learned** and **what I would add next**.

Resume bullets (fill in your real numbers)

```
Mini Redis | Java, NIO, Spring Boot, Docker                          [GitHub link]
- Built a Redis-compatible in-memory database in Java using a non-blocking
  event loop; works with the real redis-cli and redis-benchmark.
- Implemented 30+ commands, key expiry (lazy + active), lists, hashes, pub/sub
  and append-only-file persistence with configurable fsync.
- Reached X requests/sec (p99 under Y ms) on redis-benchmark, within Z% of Redis.
- Built a Spring Boot web dashboard with a live console, key browser and stats.
```

Interview questions you can now answer (simple words)

Question	Short answer
Why is Redis fast with one thread?	Data is in memory, there are no locks to wait for, and one event loop handles many clients without switching between threads.
What is lazy vs active expiry?	Lazy: check the time when a key is read. Active: a timer removes some dead keys regularly. Together they keep memory clean without slowing reads.
What is AOF and what is fsync?	AOF is a log of all changes that you replay on restart. fsync forces data to disk: 'always' is safest but slow, 'everysec' loses at most 1 second, 'no' is fastest.
What problem does partial reading solve?	TCP is a stream. A command can arrive in pieces or glued with others, so the parser must wait for more bytes and never lose data.
How does pub/sub work?	A map from channel name to subscribers. PUBLISH loops over that list and writes the message to each connection.
How would you scale it?	Add replication (copy data to other servers) and sharding (split keys across servers). This is a good 'next steps' answer.

If you fall behind: what to cut

Life happens: exams, assignments, bad days. Cut in this order and you still finish a strong project.

Cut first	Then	Never cut
LRU eviction, live chart, hashes	Pub/Sub, active expiry	Real redis-cli working, NIO loop, SET/GET/DEL, TTL, AOF, README with demo

Common mistakes (and how to avoid them)

- **Writing everything before testing.** Test after every command with redis-cli.
- **Using `synchronized` everywhere.** In Week 2 only one thread touches the data, so you do not need it.
- **Ignoring half-received commands.** It works on your laptop and breaks under the benchmark.
- **Copying code you do not understand.** In an interview you will be asked about it. Type it, break it, fix it.
- **No commits.** Commit daily. A green GitHub activity graph is itself a good signal.
- **Trying real Redis features you do not need** (clusters, Lua scripts, streams). A smaller project that is finished beats a big one that is not.

Final checklist

Done?	Item
[ ]	redis-cli connects and PING, SET, GET, DEL, INCR, EXPIRE, TTL work
[ ]	Lists and hashes work, with proper error messages
[ ]	Server stays correct when a command arrives in pieces
[ ]	Data survives a restart (AOF replay)
[ ]	Pub/Sub works between two terminals
[ ]	redis-benchmark runs and numbers are in the README
[ ]	Web dashboard shows keys, console and live stats
[ ]	docker compose up starts everything
[ ]	Unit tests pass; GitHub README has GIF, diagram, features, benchmark
[ ]	Resume bullets written; you can explain each part out loud in simple words

A last word
You do not need to understand everything on Day 1. Each day, you only need to do **one small thing** and make it work. By Day 28, those small things add up to a project most students never build. Good luck!